

# UNIVERSIDAD TÉCNICA DE MANABÍ

FACULTAD DE CIENCIAS INFORMATICAS

TECNOLOGIAS DE LA INFORMACION


**NOMBRES**

QUINTERO MENDOZA JHON ALEXANDER


**PARALELO**

“A”

**ASIGNATURA**

DESARROLLO DE SISTEMAS INFORMATICOS

**DOCENTE**

ING. JORGE ZAMBRANO

**TEMA**

## ESTUDIO DE CASO - SISTEMA DE CONTROL DE RESCATE Y ADOPCIÓN ANIMAL

## 1. Introducción

El rescate de animales requiere organizar información sobre los casos registrados, los responsables asignados y la atención que recibe cada animal. Cuando estos datos no se gestionan adecuadamente, puede resultar difícil conocer el estado de cada caso y dar seguimiento a su proceso.

Por esta razón, se propone el diseño de un sistema para gestionar los casos de rescate de perros, gatos y animales silvestres. El sistema permitirá registrar los casos, asignar responsables, actualizar estados y organizar la información relacionada con la atención veterinaria y las adopciones.

Para representar el funcionamiento y la estructura del sistema se utilizará el Lenguaje Unificado de Modelado (UML), que permite visualizar y documentar el diseño de un sistema mediante diferentes tipos de diagramas (Object Management Group [OMG], 2017). También se considerarán patrones de diseño, que ofrecen soluciones reutilizables para problemas frecuentes en el desarrollo de software (Gamma et al., 1994). Finalmente, se utilizará Gitflow para organizar los avances del diseño mediante un repositorio de GitHub.

## 2. Análisis del escenario

El sistema propuesto busca facilitar el registro y seguimiento de los animales rescatados, desde la creación del caso hasta su atención, adopción o liberación, cuando corresponda. Para definir su funcionamiento, se identifican los usuarios que interactuarán con el sistema y las funciones que deberá ofrecer. Los requerimientos permiten establecer estas necesidades y sirven como base para el diseño del software (Haigh, 2023).

### 2.1 Identificación de actores

Los actores representan a las personas que interactúan directamente con el sistema. Para este escenario se consideran los siguientes:

- **Ciudadano o reportante:** registra información sobre un animal que necesita ayuda.
- **Voluntario:** consulta los casos asignados y actualiza su seguimiento.
- **Veterinario:** registra la atención médica y los tratamientos realizados.
- **Administrador del refugio:** supervisa los casos y asigna responsables.
- **Adoptante:** consulta los animales disponibles y presenta solicitudes de adopción.

### 2.2 Requerimientos funcionales

Los requerimientos funcionales describen las acciones que el sistema debe permitir realizar (Haigh, 2023). Para este proyecto se proponen los siguientes:

- **RF01.** Registrar nuevos casos de rescate.
- **RF02.** Guardar los datos del animal, incluyendo su tipo y condición.
- **RF03.** Asignar voluntarios o veterinarios responsables.
- **RF04.** Consultar los casos registrados y sus estados.
- **RF05.** Actualizar el estado de cada caso.

- **RF06.** Registrar las atenciones veterinarias.
- **RF07.** Gestionar las solicitudes de adopción.
- **RF08.** Generar notificaciones sobre novedades importantes de los casos.

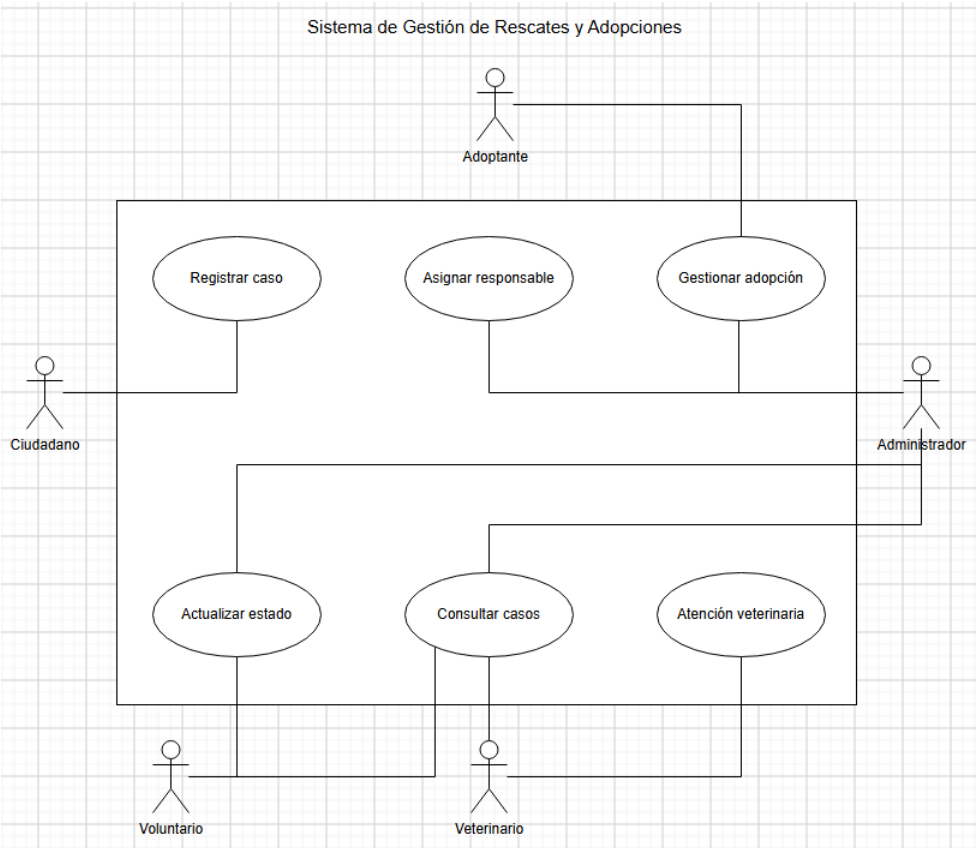
### 2.3 Requerimientos no funcionales

Los requerimientos no funcionales describen características de calidad y condiciones que debe cumplir el sistema, además de las funciones que realiza (Haigh, 2023). Para esta propuesta se consideran los siguientes:

- **RNF01. Mantenibilidad:** el diseño debe facilitar la modificación y corrección del sistema.
- **RNF02. Seguridad:** el acceso a la información debe estar limitado según el rol del usuario.
- **RNF03. Usabilidad:** las funciones deben presentarse de forma clara y sencilla.
- **RNF04. Trazabilidad:** los cambios realizados durante el diseño deben quedar registrados en el repositorio de GitHub.

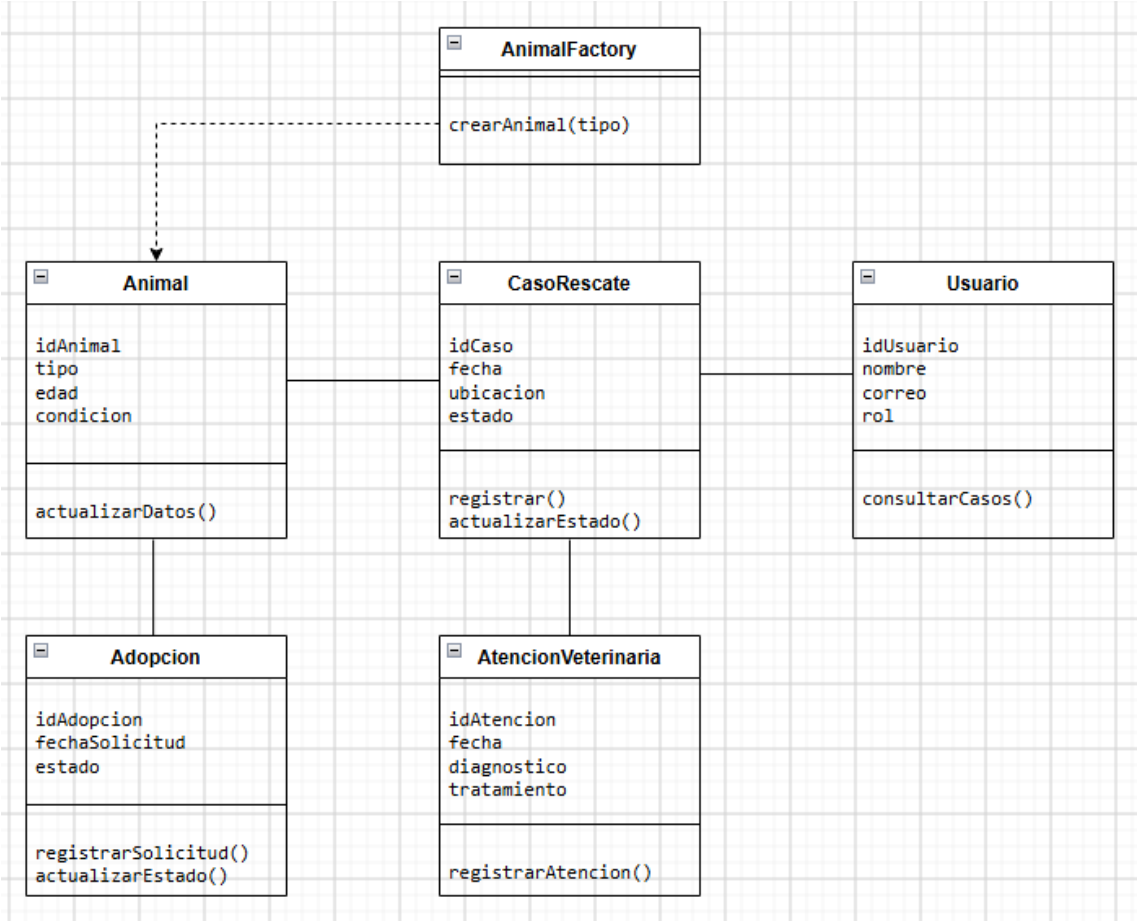
## 3. Modelado UML

El diagrama de casos de uso representa las funciones principales del sistema y su relación con los usuarios. Cada actor puede realizar determinadas acciones según su responsabilidad. Por ejemplo, el administrador asigna responsables y supervisa los casos, mientras que el veterinario registra la atención médica. Este diagrama permite comprender de forma visual cómo se utilizará el sistema.



### 3.2 Diagrama de clases UML

El diagrama de clases representa la estructura del sistema, incluyendo sus clases, atributos, métodos y relaciones. A diferencia del diagrama de casos de uso, que muestra las funciones disponibles para los usuarios, este diagrama permite observar cómo se organiza internamente el sistema (Object Management Group [OMG], 2017).




## 4. Patrones de diseño

Los patrones de diseño son soluciones reutilizables para problemas comunes al organizar un sistema de software. Gamma et al. (1994) describen patrones creacionales, estructurales y de comportamiento. Para este proyecto se proponen tres patrones, cada uno con una función diferente.

### 4.1 Factory Method (patrón creacional)

Factory Method permite separar la creación de objetos del resto del sistema (Gamma et al., 1994). En el refugio se propone utilizarlo para crear objetos que representen distintos tipos de animales, como perros, gatos y fauna silvestre. De esta manera, se puede organizar la creación de animales sin concentrar esa lógica en la clase que registra los casos.

### 4.2 Observer (patrón de comportamiento)

Observer permite notificar a otros componentes cuando ocurre un cambio en un objeto (Gamma et al., 1994). En el sistema propuesto, podría utilizarse para avisar a los voluntarios o veterinarios cuando se registre un caso nuevo o se les asigne una responsabilidad. Esto ayudaría a mantener informados a los responsables sin vincular directamente la gestión de casos con cada tipo de notificación.

### 4.3 Adapter (patrón estructural)

Adapter permite conectar componentes con interfaces diferentes mediante una clase intermediaria (Gamma et al., 1994). En este proyecto, podría utilizarse para integrar el sistema con un servicio externo de notificaciones, adaptando la forma en que el sistema solicita el envío de avisos a la interfaz que utiliza dicho servicio.

## 5. Uso de Gitflow en GitHub

Gitflow es un modelo de trabajo que permite organizar los cambios de un proyecto mediante diferentes ramas de Git. Su objetivo es mantener los avances ordenados y facilitar el registro de las modificaciones realizadas durante el desarrollo (Driessen, 2010).

Para el sistema de gestión de rescates de animales, se utilizarán las siguientes ramas:

- **main:** contendrá la versión final y estable del proyecto.
- **develop:** reunirá los avances antes de incorporarlos a la versión final.
- **feature/analisis:** se utilizará para trabajar en el análisis de actores y requisitos.
- **feature/diagramas-uml:** permitirá registrar la elaboración de los diagramas de casos de uso y clases.
- **feature/patrones:** servirá para documentar la selección y justificación de los patrones de diseño.

Los avances se registrarán mediante commits con mensajes claros, por ejemplo: “Agregar requisitos funcionales”, “Crear diagrama de casos de uso” y “Documentar patrones de diseño”. Cuando cada parte esté terminada, sus cambios se integrarán en la rama develop. Finalmente, la versión revisada se incorporará a main.

De esta manera, el repositorio de GitHub permitirá observar el progreso del trabajo y conservar un historial de los cambios realizados.

## 6. Conclusiones

El análisis realizado permitió identificar las principales funciones y los usuarios del sistema de gestión de rescates de animales. La definición de los requisitos ayuda a establecer qué debe hacer el sistema y cómo se organizará el trabajo del refugio.

Los diagramas UML permiten representar de forma visual el funcionamiento del sistema y la relación entre sus elementos. Además, los patrones Factory Method, Observer y Adapter ofrecen alternativas para organizar la creación de animales, las notificaciones y la conexión con servicios externos.

Finalmente, el uso de GitHub y del modelo Gitflow permite mantener un registro de los cambios realizados en el proyecto. Esto facilita el seguimiento de los avances y ayuda a conservar una versión organizada del trabajo.

## 7. Referencias bibliográficas


Object Management Group. (2017). *About the Unified Modeling Language specification version 2.5.1*. https://www.omg.org/spec/UML/About-UML/

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley Professional. https://www.pearson.com/en-us/subject-catalog/p/design-patterns-elements-of-reusable-object-oriented-software/P200000009480/9780201633610

Haigh, F. (2023, 5 de junio). *SRS - Software Requirements Specification*. NASA Software Engineering Handbook. https://swehb.nasa.gov/spaces/SWEHBVB/pages/32604425/SRS%2B-%2BSoftware%2BRequirements%2BSpecification

Driessen, V. (2010, 5 de enero). *A successful Git branching model*. https://nvie.com/posts/a-successful-git-branching-model/

## 8. Enlaces a repositorio GitHub y a diseños en Draw.io

- ✓ https://github.com/jquintero8667/DSI1
- ✓ https://drive.google.com/file/d/1WOj5FbgXcBA4rsA_pL_mu_jgKJk78XGE/view?usp=sharing
- ✓ https://drive.google.com/file/d/1nsaADWHY1btOPpeharqOaOrAhRX67d65/view?usp=sharing

## 9. Anexos

